

Machine Learning: The No-Nonsense (But Fun) Guide — Day 8

Advanced KNN & Optimization: Making the "Lazy Neighbor" Fast 🚀

1. The Problem: KNN Is Too Slow 🐌

Yesterday we learned that **K-Nearest Neighbors (KNN)** is a **lazy learning algorithm**.

It doesn't spend time learning a mathematical model.

Instead, it simply stores the entire training dataset.

The problem appears during prediction.

Every time a new sample arrives, KNN compares it with **every training sample**.

Time Complexity

Phase	What Happens	Time Complexity
Training	Store data	O(1) (Almost Instant)
Prediction	Calculate distance to every point	O(n × d)

Where:

- **n** = Number of samples

- d = Number of features

Imagine having **1 million samples**.

To classify just **one new sample**, KNN calculates the distance to every single point.

That makes prediction extremely slow.

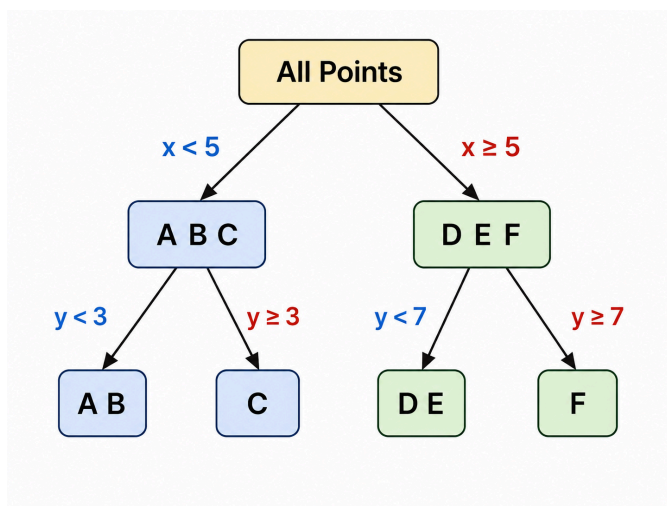
KNN: Great for small datasets. A nightmare for very large datasets.

2. KD-Tree: Split Space Like a Pro 🌳

Idea

Instead of checking every point,

split the data recursively into smaller regions.



The tree works similarly to **Binary Search**.

Instead of searching the entire dataset,

KNN traverses only the relevant branches.

Advantages

- Much faster searching
- Skip unnecessary branches
- Average search complexity:

$O(\log n)$

Implementation

```
from sklearn.neighbors import KNeighborsClassifier
```

```
knn = KNeighborsClassifier(  
    n_neighbors=5,  
    algorithm="kd_tree"  
)
```

Limitation

KD-Tree works well only for **low-dimensional data**.

Above roughly **20 features**, it becomes slower because of the **Curse of Dimensionality**.

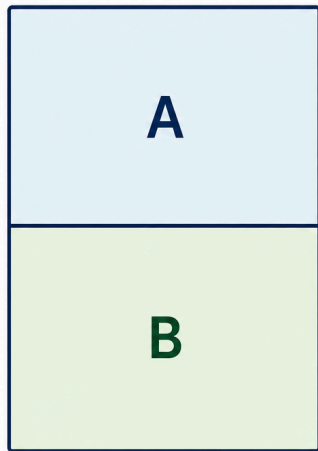
3. Ball Tree: Spheres Instead of Boxes

KD-Tree divides space using **rectangles**.

Ball Tree divides space using **hyperspheres**.

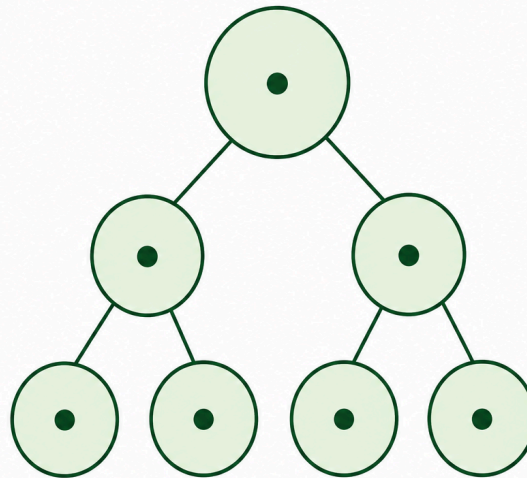
KD Tree

(Axis-Aligned Splits)



Ball Tree

(Hyperspherical Splits)



Advantages

- Better for higher-dimensional data
- Supports many distance metrics
- Performs well when data is clustered

Implementation

```
knn = KNeighborsClassifier(  
    n_neighbors=5,  
    algorithm="ball_tree"  
)
```

4. Approximate Nearest Neighbors (ANN)

Idea

Instead of finding the **exact nearest neighbors**,

find neighbors that are **close enough**.

This dramatically improves speed.

Popular ANN Libraries

Method	How It Works	Speedup
LSH	Hash similar points	10–100×
Annoy	Random projection trees	100–1000×
FAISS	Quantization + GPU	1000×+
HNSW	Small-world graphs	100–1000×

Trade-off:

- Slightly lower accuracy
- Massive speed improvement

Example

```
from annoy import AnnoyIndex
```

```
f = 128
```

```
t = AnnoyIndex(f, "euclidean")
```

```
for i in range(10000):  
    t.add_item(i, vector[i])
```

```
t.build(10)
```

```
neighbors = t.get_nns_by_vector(query_vector, 5)
```

ANN Philosophy

"Close enough" is often better than "exact" when speed matters.

5. Curse of Dimensionality 🤔

Adding more features sounds useful.

Unfortunately,

it makes KNN worse.

What Happens?

Number of Features	Effect
2–10	Distances work well
10–20	Performance starts degrading
20–50	Most points become equally far
50+	Distance loses meaning

As dimensions increase,

the nearest and farthest neighbors become almost the same distance away.

KNN can no longer distinguish similar points.

6. PCA with KNN

Principal Component Analysis (PCA)

reduces the number of features while preserving important information.

100 Features



PCA



10 Principal Components



KNN

Benefits

- Faster prediction
- Less memory
- Better distance calculations
- Reduced Curse of Dimensionality

Implementation

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=10)
```

```
X_reduced = pca.fit_transform(X_scaled)
```

```
knn = KNeighborsClassifier(n_neighbors=5)
```

```
knn.fit(X_reduced, y)
```

7. Feature Scaling (Mandatory)

Distance-based algorithms require features to be on similar scales.

Without Scaling

House Size

1000
2000
3000
5000

Bedrooms

1
2
3
4

House size completely dominates the distance.

Bedrooms contribute almost nothing.

With Scaling

House Size

0.1
0.3

0.5
0.8

Bedrooms

0.2
0.4
0.6
0.8

Now every feature contributes equally.

StandardScaler

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

MinMaxScaler

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

⚠ **Always scale your data before using KNN.**

8. Choosing the Best K using Cross Validation

Never guess the value of **K**.

Let the data decide.

Example

```
from sklearn.model_selection import cross_val_score
```

```
scores = []
```

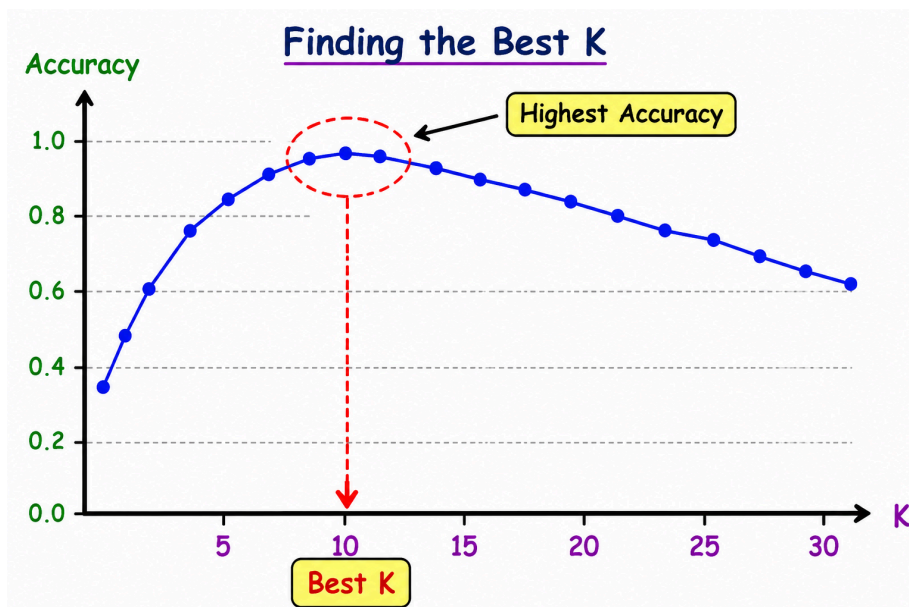
```
for k in range(1,31):
```

```
    model = KNeighborsClassifier(n_neighbors=k)
```

```
    score = cross_val_score(model, X_scaled, y, cv=5)
```

```
    scores.append(score.mean())
```

Plot the scores.



Choose the value with the highest validation accuracy.

9. Choosing Distance Metrics

Data Type	Best Metric
Continuous Data	Euclidean
City Blocks	Manhattan
Mixed Data	Gower
Text Data	Cosine Similarity
Binary Data	Hamming

Example

```
KNeighborsClassifier(metric="euclidean")
```

```
KNeighborsClassifier(metric="manhattan")
```

```
KNeighborsClassifier(metric="cosine")
```

```
KNeighborsClassifier(metric="minkowski", p=3)
```

10. Weighted KNN

Standard KNN treats every neighbor equally.

Weighted KNN gives **more importance to closer neighbors**.

Distance = 1

Weight = 1.0

Distance = 2

Weight = 0.5

Distance = 5

Weight = 0.2

Implementation

```
KNeighborsClassifier(  
  
    n_neighbors=5,  
  
    weights="distance"  
)
```

Use Weighted KNN whenever nearby samples should influence predictions more strongly.

11. KNN Optimization Cheat Sheet

Problem	Solution
Large dataset	KD Tree / Ball Tree
Huge dataset	FAISS / Annoy
Too many features	PCA
Features on different scales	StandardScaler
Unknown K	Cross Validation

Noisy data	Increase K
Better predictions	Weighted KNN
Binary data	Hamming Distance

12. Optimizing KNN for One Million Samples

A practical pipeline:

Raw Dataset



StandardScaler



PCA



Ball Tree / FAISS



Weighted KNN



Prediction

Full Pipeline

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
```

```
scaler = StandardScaler()
```

```
pca = PCA(n_components=50)
```

```
knn = KNeighborsClassifier(
```

```
    n_neighbors=10,
```

```
    algorithm="ball_tree",
```

```
    weights="distance",
```

```
    n_jobs=-1
```

```
)
```

```
X_scaled = scaler.fit_transform(X)
```

```
X_reduced = pca.fit_transform(X_scaled)
```

```
knn.fit(X_reduced, y)
```

13. Practical Tips

Tip	Why?
Always scale features	Distance depends on scale

Start with $K \approx \sqrt{n}$	Good starting point
Use odd K	Avoid ties
Try multiple distance metrics	Performance varies
Handle missing values	KNN cannot process NaN
Consider ANN in production	Huge speed improvement

14. Interview Questions

1. What is the Curse of Dimensionality?
 2. Explain KD-Tree.
 3. Difference between KD-Tree and Ball Tree.
 4. Why is PCA used before KNN?
 5. What are Approximate Nearest Neighbors?
 6. Explain Weighted KNN.
 7. Which distance metric is best for text data?
 8. How would you optimize KNN for one million samples?
 9. What is Cross Validation?
 10. Why is feature scaling important?
-

Happy Learning! 🚀